



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 05

NOMBRE COMPLETO: Lechuga Castillo Shareny Ixchel

N° de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 25-09-2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

ACTIVIDADES:

- 1.- Importar su modelo de coche propio dentro del escenario a una escala adecuada.
- 2.- Importar sus 4 llantas y acomodarlas jerárquicamente, agregar el mismo valor de rotación a las llantas para que al presionar puedan rotar hacia adelante y hacia atrás.
- 3.- Importar el cofre del coche, acomodarlo jerárquicamente y agregar la rotación para poder abrir y cerrar.
- 4.- Agregar traslación con teclado para que pueda avanzar y retroceder de forma independiente

El código que tenemos es el siguiente:

En practica05.cpp tenemos

```
23  #include "Window.h"
24  #include "Mesh.h"
25  #include "Shader_m.h"
26  #include "Camera.h"
27  #include "Sphere.h"
28  #include "Model.h"
29  #include "Skybox.h"
30
31  const float toRadians = 3.14159265f / 180.0f;
32  //float angulocola = 0.0f;
33  Window mainWindow;
34  std::vector<Mesh*> meshList;
35  std::vector<Shader> shaderList;
36
37  Camera camera;
38  Model BaseCoche;
39  Model CofreCoche;
40  Model LLantaDelanteraIzquierda;
41  Model LLantaDelanteraDerecha;
42  Model LLantaTraseraIzquierda;
43  Model LLantaTraseraDerecha;
44  Skybox skybox;
45
46  //Sphere cabeza = Sphere(0.5, 20, 20);
47  GLfloat deltaTime = 0.0f;
48  GLfloat lastTime = 0.0f;
49  static double limitFPS = 1.0 / 60.0;
50
51
52  // Vertex Shader
53  static const char* vShader = "shaders/shader_m.vert";
54
55  // Fragment Shader
56  static const char* fShader = "shaders/shader_m.frag";
```

```

108 int main()
109 {
110     mainWindow = Window(1366, 768); // 1280, 1024 or 1024, 768
111     mainWindow.Initialise();
112
113     CreateObjects();
114     CreateShaders();
115
116     camera = Camera(glm::vec3(0.0f, 0.5f, 7.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 1.0f);
117
118     BaseCoche = Model();
119     BaseCoche.LoadModel("Models/Coche_Base.obj");
120     CofreCoche = Model();
121     CofreCoche.LoadModel("Models/cofre_Coche.obj");
122     LLantaDelanteraIzquierda = Model();
123     LLantaDelanteraIzquierda.LoadModel("Models/LLanta_Delantera_Izquierda.obj");
124     LLantaDelanteraDerecha = Model();
125     LLantaDelanteraDerecha.LoadModel("Models/LLanta_Delantera_Derecha.obj");
126     LLantaTraseraIzquierda = Model();
127     LLantaTraseraIzquierda.LoadModel("Models/LLanta_Trasera_Izquierda.obj");
128     LLantaTraseraDerecha = Model();
129     LLantaTraseraDerecha.LoadModel("Models/LLanta_Trasera_Derecha.obj");
130     std::vector<std::string> skyboxFaces;
131     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_rt.tga");
132     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_lf.tga");
133     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_dn.tga");
134     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_up.tga");
135     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_bk.tga");
136     skyboxFaces.push_back("Textures/Skybox/cupertin-lake_ft.tga");
137
138     skybox = Skybox(skyboxFaces);
139
140     GLuint uniformProjection = 0, uniformModel = 0, uniformView = 0, uniformEyePosition = 0,
141         uniformSpecularIntensity = 0, uniformShininess = 0;
142     GLuint uniformColor = 0;
143     glm::mat4 projection = glm::perspective(45.0f, (GLfloat)mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 1000.0f);

```

```

145     glm::mat4 model(1.0);
146     glm::mat4 modelaux(1.0);
147     glm::mat4 modelaux2(1.0); // para evitar rotaciones que no deben de en las llantas usaremos una
148     //compartida de las 4 llantas haga que se rote fuera del propio eje de la llanta
149     glm::vec3 color = glm::vec3(1.0f, 1.0f, 1.0f);
150
151     ///Loop mientras no se cierra la ventana
152     while (!mainWindow.getShouldClose())
153     {
154         GLfloat now = glfwGetTime();
155         deltaTime = now - lastTime;
156         deltaTime += (now - lastTime) / limitFPS;
157         lastTime = now;
158
159         //Recibir eventos del usuario
160         glfwPollEvents();
161         camera.keyControl(mainWindow.getKeys(), deltaTime);
162         camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());
163
164         // Clear the window
165         glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
166         glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
167         //Se dibuja el Skybox
168         skybox.DrawSkybox(camera.calculateViewMatrix(), projection);
169
170         shaderList[0].UseShader();
171         uniformModel = shaderList[0].GetModelLocation();
172         uniformProjection = shaderList[0].GetProjectionLocation();
173         uniformView = shaderList[0].GetViewLocation();
174         uniformColor = shaderList[0].getColorLocation();
175
176         glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
177         glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
178         // INICIA DIBUJO DEL PISO
179         color = glm::vec3(0.5f, 0.5f, 0.5f); //piso de color gris
180         model = glm::mat4(1.0);
181         model = glm::translate(model, glm::vec3(0.0f, -2.0f, 0.0f));
182         model = glm::scale(model, glm::vec3(30.0f, 1.0f, 30.0f));
183         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));

```

```

183 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
184 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
185 meshList[2]->RenderMesh();
186
187 //-----INICIA DIBUJO DE NUESTROS DEMÁS OBJETOS-----
188 // NOTA IMPORTANTE AQUI EL EJE X ES EL EJE Z POR CULPA DEL MODELO
189 // Cuerpo carro
190 model = glm::mat4(1.0f);
191 model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 1.0f, 0.0f));
192 model = glm::translate(model, glm::vec3(0.0f, -1.5f, mainWindow.getDesplazamiento()));
193
194 modelaux = model;
195 model = glm::scale(model, glm::vec3(.05f, .05f, .05f));
196 color = glm::vec3(0.529f, 0.808f, 0.922f); // Azul cielo
197 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
198 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
199 BaseCoche.RenderModel();
200
201 //-----Cofre carro-----
202 model = modelaux;
203 /*
204 Debido a que se rota -90 grados los ejes quedan de la siguiente manera:
205 eje z pasa a ser eje x
206 eje x de forma positiva pasa a ser -z y negativo z
207 */
208 model = glm::translate(model, glm::vec3(0.0f, 3.4f, -4.7f));
209 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion1()), glm::vec3(1.0f, 0.0f, 0.0f));
210 //modelaux = model;
211 model = glm::scale(model, glm::vec3(.05f, .05f, .05f));
212 color = glm::vec3(0.784f, 0.635f, 0.784f); // Lila
213 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
214 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
215 CofreCoche.RenderModel();

```



```

217 // -----PARA LAS LLANTAS -----
218 // LLantaDelanteraDerecha
219 model = modelaux;
220 model = glm::translate(model, glm::vec3(3.5f, 1.0f, -5.2f));
221 modelaux2 = model;
222 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
223 modelaux = model;
224 model = glm::scale(model, glm::vec3(.05f, .05f, .05f));
225 color = glm::vec3(0.0f, 0.0f, 0.0f); // Negro
226 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
227 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
228 LLantaDelanteraDerecha.RenderModel();
229
230 // LLantaDelanteraDerecha
231 model = modelaux;
232 model = glm::translate(model, glm::vec3(-7.5f, 0.0f, 0.0f));
233 model = glm::scale(model, glm::vec3(.05f, .05f, .05f));
234 color = glm::vec3(0.0f, 0.0f, 0.0f); // Negro
235 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
236 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
237 LLantaDelanteraIzquierda.RenderModel();
238
239 // LLantaTraseraDerecha
240 model = modelaux2;
241 model = glm::translate(model, glm::vec3(0.0f, 0.0f, 11.5f));
242 model = glm::rotate(model, glm::radians(mainWindow.getarticulacion2()), glm::vec3(1.0f, 0.0f, 0.0f));
243 modelaux2 = model;
244 model = glm::scale(model, glm::vec3(.05f, .05f, .05f));
245 color = glm::vec3(0.0f, 0.0f, 0.0f); // Negro
246 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
247 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
248 LLantaTraseraDerecha.RenderModel();

```

```

250 // LLantaTraseraIzquierda
251 model = modelaux2;
252 model = glm::translate(model, glm::vec3(-7.5f, 0.0f, 0.0f));
253 model = glm::scale(model, glm::vec3(.05f, .05f, .05f));
254 color = glm::vec3(0.0f, 0.0f, 0.0f); // Negro
255 glUniform3fv(uniformColor, 1, glm::value_ptr(color));
256 glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
257 LLantaTraseraIzquierda.RenderModel();
258
259 glUseProgram(0);
260
261 mainWindow.swapBuffers();
262 }
263
264 return 0;
265 }

```

En el Windows.h

```

1 #pragma once
2 #include<stdio.h>
3 #include<glew.h>
4 #include<glfw3.h>
5
6 class Window
7 {
8 public:
9     Window();
10    Window(GLint windowHeight, GLint windowHeight);
11    int Initialise();
12    GLfloat getBufferWidth() { return bufferWidth; }
13    GLfloat getBufferHeight() { return bufferHeight; }
14    bool getShouldClose() {
15        return glfwWindowShouldClose(mainWindow);
16    }
17    bool* getsKeys() { return keys; }
18    GLfloat getXChange();
19    GLfloat getYChange();
20    void swapBuffers() { return glfwSwapBuffers(mainWindow); }
21    GLfloat getrotay() { return rotay; }
22    GLfloat getrotax() { return rotax; }
23    GLfloat getrotaz() { return rotaz; }
24    GLfloat getarticulacion1() { return articulacion1; }
25    GLfloat getarticulacion2() { return articulacion2; }
26    GLfloat getarticulacion3() { return articulacion3; }
27    GLfloat getarticulacion4() { return articulacion4; }
28    GLfloat getarticulacion5() { return articulacion5; }
29    GLfloat getarticulacion6() { return articulacion6; }
30    //Definiendo más articulaciones
31    GLfloat getarticulacion7() { return articulacion7; }
32    GLfloat getarticulacion8() { return articulacion8; }
33    GLfloat getarticulacion9() { return articulacion9; }
34
35    GLfloat getDesplazamiento() { return Desplazamiento; }
36
37    GLfloat posicionCarroZ; // Nueva variable para la posición del coche en el eje Z
38
39    GLfloat getPosicionCarroZ() { return posicionCarroZ; }

```

```

41     ~Window();
42 private:
43     GLFWwindow *mainWindow;
44     GLint width, height;
45     GLfloat rotax, rotay, rotaz, articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6, articulacion7, articulacion8, articulacion9, Desplazamiento;
46     bool keys[1024];
47     GLint bufferWidth, bufferHeight;
48     GLfloat lastX;
49     GLfloat lastY;
50     GLfloat xChange;
51     GLfloat yChange;
52     bool mouseFirstMoved;
53     void createCallbacks();
54     static void ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode);
55     static void ManejaMouse(GLFWwindow* window, double xPos, double yPos);
56 };

```

En el Window.cpp

```

1      #include "Window.h"
2
3      Window::Window()
4      {
5          width = 800;
6          height = 600;
7          for (size_t i = 0; i < 1024; i++)
8          {
9              keys[i] = 0;
10         }
11     }
12     Window::Window(GLint windowWidth, GLint windowHeight)
13     {
14         width = windowWidth;
15         height = windowHeight;
16         rotax = 0.0f;
17         rotay = 0.0f;
18         rotaz = 0.0f;
19         Desplazamiento = 0.0f; // Variable para controlar la dirección
20         articulacion1 = 0.0f;
21         articulacion2 = 0.0f;
22         articulacion3 = 0.0f;
23         articulacion4 = 0.0f;
24         articulacion5 = 0.0f;
25         articulacion6 = 0.0f;
26         articulacion7 = 0.0f;
27         articulacion8 = 0.0f;
28
29         for (size_t i = 0; i < 1024; i++)
30         {
31             keys[i] = 0;
32         }
33     }
34     int Window::Initialise()
35     {
36         //Iniciación de GLFW
37         if (!glfwInit())
38         {
39             printf("Falló inicializar GLFW");
40             glfwTerminate();
41
42             glfwTerminate();
43             return 1;
44         }
45         //Asignando variables de GLFW y propiedades de ventana
46         glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, 4);
47         glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, 3);
48         //para solo usar el core profile de OpenGL y no tener retrocompatibilidad
49         glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
50         glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
51
52         //CREAR VENTANA
53         mainWindow = glfwCreateWindow(width, height, "Practica 05:Optimización y Carga de Modelos", NULL, NULL);
54
55         if (!mainWindow)
56         {
57             printf("Fallo en crearse la ventana con GLFW");
58             glfwTerminate();
59             return 1;
60         }
61         //Obtener tamaño de Buffer
62         glfwGetFramebufferSize(mainWindow, &bufferWidth, &bufferHeight);
63
64         //asignar el contexto
65         glfwMakeContextCurrent(mainWindow);
66
67         //MANEJAR TECLADO y MOUSE
68         createCallbacks();
69
70         //permitir nuevas extensiones
71         glewExperimental = GL_TRUE;
72
73         if (glewInit() != GLEW_OK)
74         {
75             printf("Falló inicialización de GLEW");
76             glfwDestroyWindow(mainWindow);
77             glfwTerminate();
78             return 1;
79         }

```

```

77         return 1;
78     }
79
80     glEnable(GL_DEPTH_TEST); //HABILITAR BUFFER DE PROFUNDIDAD
81     // Asignar valores de la ventana y coordenadas
82
83     //Asignar Viewport
84     glViewport(0, 0, bufferWidth, bufferHeight);
85     //Callback para detectar que se está usando la ventana
86     glfwSetWindowUserPointer(mainWindow, this);
87 }
88
89 void Window::createCallbacks()
90 {
91     glfwSetKeyCallback(mainWindow, ManejaTeclado);
92     glfwSetCursorPosCallback(mainWindow, ManejaMouse);
93 }
94
95 GLfloat Window::getXChange()
96 {
97     GLfloat theChange = xChange;
98     xChange = 0.0f;
99     return theChange;
100 }
101
102 GLfloat Window::getYChange()
103 {
104     GLfloat theChange = yChange;
105     yChange = 0.0f;
106     return theChange;
107 }
108
109 void Window::ManejaTeclado(GLFWwindow* window, int key, int code, int action, int mode)
110 {
111     Window* theWindow = static_cast<Window*>(glfwGetWindowUserPointer(window));
112
113     if (key == GLFW_KEY_ESCAPE && action == GLFW_PRESS)
114     {
115         glfwSetWindowShouldClose(window, GL_TRUE);
116     }
117
118     if (key == GLFW_KEY_E)
119     {
120         theWindow->rotax += 10.0;
121     }
122     if (key == GLFW_KEY_R)
123     {
124         theWindow->rotay += 10.0; //rotar sobre el eje y 10 grados
125     }
126     if (key == GLFW_KEY_T)
127     {
128         theWindow->rotaz += 10.0;
129     }
130     if (key == GLFW_KEY_F)
131     {
132         if (theWindow->articulacion1 > 30) {
133             }
134         else {
135             theWindow->articulacion1 += 10.0;
136         }
137     }
138
139     if (key == GLFW_KEY_G)
140     {
141         if (theWindow->articulacion1 == 0) {
142             }
143         else {
144             theWindow->articulacion1 -= 10.0;
145         }
146     }
147 }
148
149

```

```

151         if (key == GLFW_KEY_H)
152         {
153             if (theWindow->Desplazamiento < -40) {
154             }
155             else {
156                 theWindow->articulacion2 -= 10.0;
157                 theWindow->Desplazamiento -= 1;
158             }
159         }
160     }
161     if (key == GLFW_KEY_J)
162     {
163         if (theWindow->Desplazamiento > 40) {
164         }
165         else {
166             theWindow->articulacion2 += 10.0;
167             theWindow->Desplazamiento += 1;
168         }
169     }
170     if (key == GLFW_KEY_K)
171     {
172         theWindow->articulacion5 += 10.0;
173     }
174     if (key == GLFW_KEY_L)
175     {
176         theWindow->articulacion6 += 10.0;
177     }
178     if (key == GLFW_KEY_Z)
179     {
180         theWindow->articulacion7 += 10.0;
181     }
182     if (key == GLFW_KEY_X)
183     {
184         theWindow->articulacion8 += 10.0;
185     }

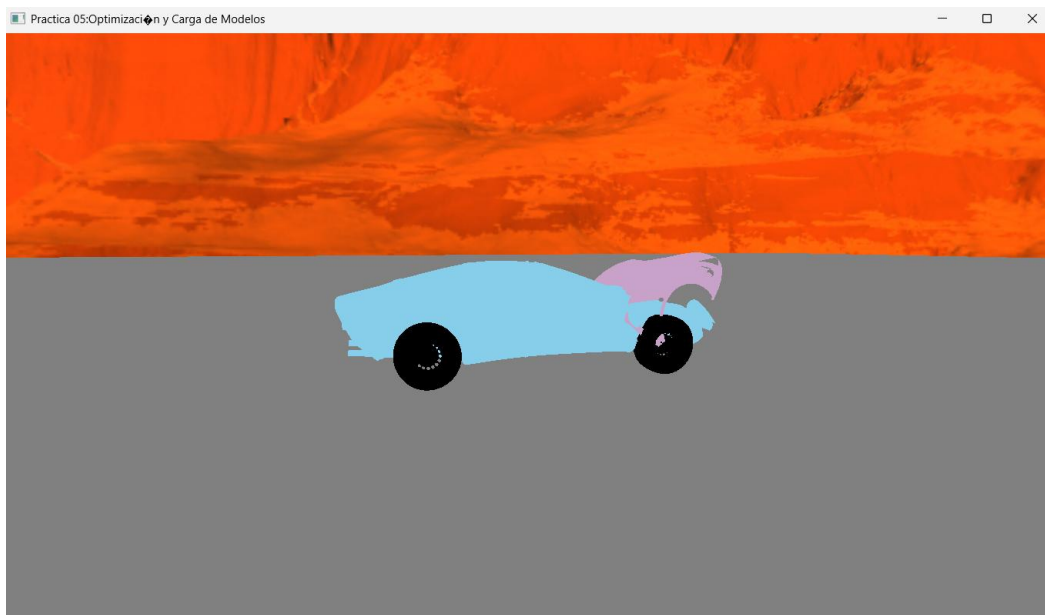
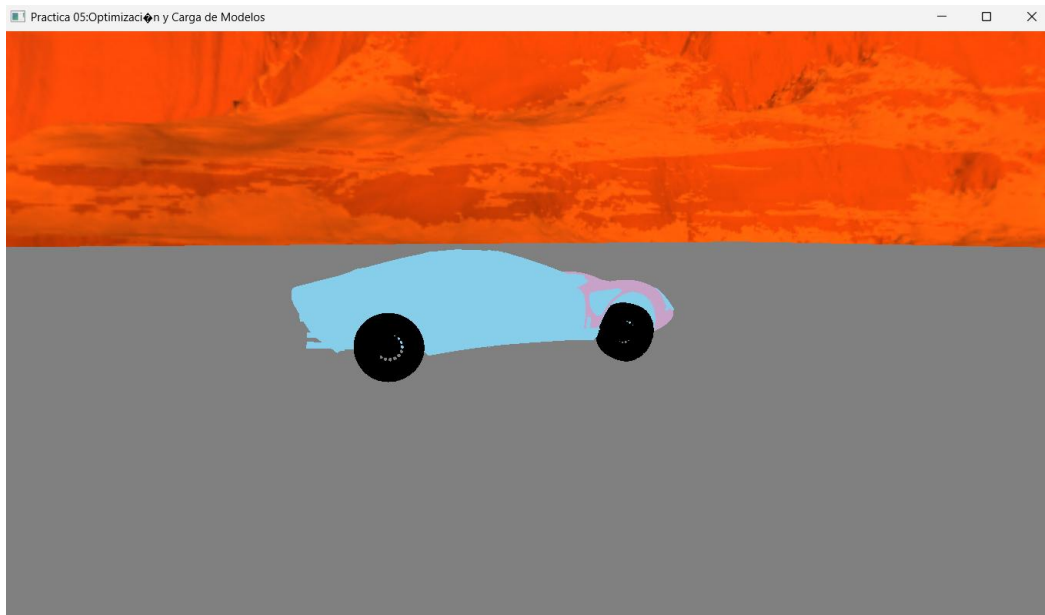
```

```

188     if (key == GLFW_KEY_D && action == GLFW_PRESS)
189     {
190         const char* key_name = glfwGetKeyName(GLFW_KEY_D, 0);
191         //printf("se presiono la tecla: %s\n", key_name);
192     }
193 }
194
195 if (key >= 0 && key < 1024)
196 {
197     if (action == GLFW_PRESS)
198     {
199         theWindow->keys[key] = true;
200         //printf("se presiono la tecla %d'\n", key);
201     }
202     else if (action == GLFW_RELEASE)
203     {
204         theWindow->keys[key] = false;
205         //printf("se solto la tecla %d'\n", key);
206     }
207 }
208
209 void Window::ManejaMouse(GLFWwindow* window, double xPos, double yPos)
210 {
211     Window* theWindow = static_cast<Window*>(glfwGetWindowUserPointer(window));
212
213     if (theWindow->mouseFirstMoved)
214     {
215         theWindow->lastX = xPos;
216         theWindow->lastY = yPos;
217         theWindow->mouseFirstMoved = false;
218     }
219
220     theWindow->xChange = xPos - theWindow->lastX;
221     theWindow->yChange = theWindow->lastY - yPos;
222
223     theWindow->lastX = xPos;
224     theWindow->lastY = yPos;
225 }

```

Los ejecutables son:



PROBLEMAS

En esta ocasión casi todo se me complicó, la verdad fue más difícil para mí usar blender que 3dsmax, no me salió al cien el proceso que hicimos en clase y se me complicó demasiado lograr separar cada llanta y el cofre, la verdad ahora si le batallé demasiado, pero con poquito de paciencia y con mucha ayuda de videos y de IA pude lograr separar cada parte del modelo.

Conclusión:

- a. Los ejercicios del reporte: En esta práctica se me complico mucho separar cada parte del modelo original, ya que no entendí muy bien blender.
- b. Comentarios generales: La explicación fue muy rápida y me perdí mucho en clase, considero que se pudo haber explicado con más tiempo y así resolver las dudas en clase.
- c. Conclusión: Creo que si fue una práctica muy difícil pero sé que debo estudiar mucho más blender y poder entenderlo mejor, así tendré mis prácticas mucho más estéticas y podré realizarlas más rápido de lo que puedo hacerlas ahorita

Bibliografía en formato APA

- Introducción — *Blender Manual*. (2025). Blender.org.
https://docs.blender.org/manual/es/2.82/getting_started/about/introduction.html
- (2025). Youtube.com. <https://www.youtube.com/watch?v=O-tV7uBf5LI>